LONDON METROPOLITAN UNIVERSITY

ITAHARI INTERNATIONAL COLLEGE

Module Code & Module Title
CC5051NT Databases

Assessment Weightage & Type
60% Individual Coursework

Year and Semester
2025-26 Autumn (Sem 1)
Credit: 15

Student Name: Sadikshya Karki
Group: L2C4
London Met ID: 24046034
Submitted To: Mr Ajayraj Bhattarai
Assignment Due Date: Sunday, January 18, 2026
Assignment Submission Date: Sunday, January 18, 2026

I confirm that I understand my coursework needs to be submitted online via mySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

Table Of Contents

1. Introduction 1
    1.1. Introduction of the Business and its Forte 1
        Aim 3
        Objectives 3
        Introduction to Module 4
    1.2. Description of Current Business Activities and Operations 5
    1.3. Business Rules Derived from Operational Procedures 7
2. Initial ERD 9
    2.1. Identification of Entities and Attributes (with Primary Keys) 9
        2.1.1. List of Entities and Attributes (with Primary Keys) 11
    2.2. Initial Entity-Relationship Diagram (Before Normalization) 12
        2.2.1. Relationship between Entities with Maximum and Minimum Cardinalities 12
        2.2.2. Initial Entity Relationship Diagram 13
3. Normalization 14
    3.1. Unnormalized Form (UNF) 15
    3.2. First Normal Form (1NF) 17
    3.3. Second Normal Form (2NF) 18
    3.4. Third Normal Form (3NF) 20
4. Data Dictionary and Final ERD 23
    4.1. List of Entities after Normalization 23
    4.2. Final ERD 27
5. Database Implementation 28
    5.1. Creating Relations and Tables 28
    5.2. Inserting Rows in Tables 30
6. Database Quering 34
    6.1. Information Query 34
    6.2. Transaction Query 36
7. Critical Evaluation 38
    7.1. Critical Evaluation of Module, Its Usage and Relation with Other Subject 38

7.2. Critical Assessment of Coursework 38
8. Drop Query and Database Dump file Creation 40
8.1. Drop Query 40
8.2. Dump file Creation 41
9. Conclusion 42
10. References 43

Table Of Figures

Figure 1 : Relationship between Entities 12
Figure 2 : Initial Entity Relationship Diagram 13
Figure 3 : Final Entity Relationship Diagram 27
Figure 4 : Passenger Table Created 28
Figure 5 : Route Table Created 28
Figure 6 : Bus Table Created 28
Figure 7 : Staff Table Created 28
Figure 8 : Trip Table Created 29
Figure 9 : Trip_Staff Table Created 29
Figure 10 : Booking Table Created 29
Figure 11 : Ticket Table Created 29
Figure 12 : Passenger Table Created 30
Figure 13 : Rows Inserted into Passenger Table 30
Figure 14 : Rows Inserted into Route Table 30
Figure 15 : Rows Inserted into Bus Table 31
Figure 16 : Rows Inserted into Staff Table 31
Figure 17 : Rows Inserted into Trip Table 31
Figure 18 : Rows Inserted into Trip_Staff Table 32
Figure 19 : Rows Inserted into Booking Table 32
Figure 20 : Rows Inserted into Ticket Table 32
Figure 21 : Rows Inserted into Payment Table 33
Figure 22 : Information Query 1 34
Figure 23 : Information Query 2 34
Figure 24 : Information Query 3 34
Figure 25 : Information Query 4 35
Figure 26 : Information Query 5 35
Figure 27 : Transaction Query 1 36
Figure 28 : Transaction Query 2 36
Figure 29 : Transaction Query 3 36
Figure 30 : Transaction Query 4 37
Figure 31 : Transaction Query 5 37

Figure 32 : Passenger Table Dropped 40
Figure 33 : Route Table Dropped 40
Figure 34 : Bus Table Dropped 40
Figure 35 : Staff Table Dropped 40
Figure 36 : Trip Table Dropped 40
Figure 37 : Trip_Staff Table Dropped 40
Figure 38 : Booking Table Dropped 40
Figure 39 : Ticket Table Dropped 41
Figure 40 : Payment Table Dropped 41
Figure 41 : Dump File Created 41

Table Of Tables

Table 1 : Trip Attributes 9
Table 2 : Passenger Attributes 10
Table 3 : Booking Attributes 10
Table 4 : Ticket Attributes 10
Table 5 : Entities and Attributes 11
Table 6 : Route Table 23
Table 7 : Bus Table 23
Table 8 : Staff Table 24
Table 9 : Trip Table 24
Table 10 : Trip_Staff Table 24
Table 11 : Passenger Table 25
Table 12 : Booking Table 25
Table 13 : Ticket Table 26
Table 14 : Payment Table 26

1. Introduction

1.1. Introduction of the Business and its Forte

SajiloYatra is a new intercity travel service based in Pokhara and was founded by Mr. Kiran Thapa. The company aims to transform long-distance travel in Nepal using digital solutions and organized management practices to deal with the issues of the transportation industry that have existed for a long time.

As the demand of travel has been increasing in major cities like Kathmandu, Pokhara, Chitwan, and Butwal, travelers have faced challenges that affect their travel experience. Such challenges are the lack of consistency in the quality of services provided, poor process of booking, and unreliable information regarding trip schedules and availability. Intercity transport services in Nepal face operational inefficiencies, which have led to poor customer experience and a lack of confidence in the services provided by the transportation sector.

SajiloYatra aims to change the intercity transportation sector by offering a more professional, reliable and comfortable traveling experience. The strategy of the company focuses on the use of digital innovation and centralized operation management as a key to providing the best travel experience to the passengers, starting with the booking process to the arrival.

The competitive advantages and operational strengths (forte) of SajiloYatra are built on a foundation of modern technology and systematic business processes:

a. Online Booking and Management System

This centralized system enables passengers to make bookings online from anywhere, view real-time seat availability, reserve preferred classes and get immediate booking confirmations.

b. Complete Integration of Business Components

The company incorporates all the necessary operational elements, such as routes, buses, trips, passengers, staff, tickets, and payments, into a single unified platform.

c. Resolution of Critical Industry Challenges

i. Inefficient Booking Systems: By implementing real-time data synchronization, the company manually erases any mistakes in booking, minimizes the processing time, and immediately confirms the passengers.

ii. Overbooked Trips: The centralized database monitors the availability of seats in real-time and prevents multiple bookings to the same seat.

iii. Lack of Centralized Management: The centralization allows the management to see the operations, decide based on facts and ensure the same set of service standard on all routes.

d. Enhanced Communication and Coordination

The two-way communication keeps everyone updated with correct, real-time information that lessens confusion and increases the reliability of the services.

e. Operational Efficiency using Technology

SajiloYatra uses the modern technological infrastructure to enhance all its operations.

f. Secure and Transparent Financial management

The ticketing and payment system has a strong security feature and is fully transparent.

g. Quality Control and Optimized Resource Management

The centralized system offers the management with effective resource allocation and service quality monitoring tools.

With these strengths, SajiloYatra positions itself as a modern, technology-driven transportation service that creates new standards of intercity transportation in Nepal.

Aim

The main aim of SajiloYatra is to transform intercity travel in Nepal by providing a reliable and efficient transportation service that uses modern technology to solve the booking and management problems that have troubled the industry for years.

Objectives

a. To create a centralized online booking and management system where passengers can easily book tickets, view real-time seat availability and get instant confirmations.

b. To solve overbooking problems by using a database that accurately keeps the record of available seats and prevents the same seat from being booked more than once.

c. To bring all aspects of the business together in one system, connecting routes, buses, trip schedules, passenger information, staff assignments, tickets, and payments for smoother operations.

d. To build a secure payment and ticketing system that handles all transaction safely and keep clear records of every payment made by passengers.

e. To help management make better decisions by providing detailed reports on operational performance and the travel pattern followed by the passengers.

Introduction to Module

A database can be defined as a set of related information that is arranged to ensure that it is easy to store, retrieve, and use. It serves as a digital file system in which everything is in its appropriate place and can be retrieved whenever required. A database is used to store all information in a single central location rather than storing it separately in various files or papers. Databases help eliminate duplicate information, allow multiple people to work with data at the same time, protect against accidental data loss, and ensure that only valid information gets entered into the system. For example, SajiloYatra uses a database system to keep passenger bookings, trip details, bus and staff information, tickets, and payments well organized in one place (Silberschatz et al., 2020).

A database management system is the software that controls and manages the database and performs all the technical work in the background. It takes care of all the complex processes of storing and retrieving information so that the users do not have to worry about technical details. The DBMS allows users to create databases, update information, retrieve specific data, and restrict access to sensitive information to authorized users only. Oracle SQL, MySQL, and PostgreSQL are used by many businesses to manage their databases because they make operations more reliable and easily manageable. The DBMS works in layers where the bottom layer deals with the physical storage of the data saved on computers, the middle layer organizes the data and shows how different pieces relate to each other, and

the top layer provides customized views to different users based on the information needed. In SajiloYatra's intercity travel system, the system makes sure that each booking matches its trip and bus. It also keeps all tickets and payment records in order (Silberschatz et al., 2020).

1.2. Description of Current Business Activities and Operations

SajiloYatra is an intercity transportation service company, based in Pokhara, Nepal that operates an organized system to manage its intercity travel business, covering the main aspects of the service linking all major cities of Nepal like Kathmandu, Pokhara, Chitwan and Butwal. The company solves significant problems in Nepal's travel sector, such as poor booking systems, overbooking, and absence of centralized management by using a modern and integrated database system to control all operations of the organization.

a. Route and Trip Management

SajiloYatra operates different intercity services between major cities in Nepal. Multiple trips are made along each route over a day and there are distinct schedules of departures and arrivals that help travelers to make plans. Each trip has a fixed route which is travelled on a particular date, offering guaranteed transportation to the passengers.

b. Buses and Maintenance Operations

The company allocates several buses to every route to ensure that they offer continuous service. Regular maintenance check-ups and unpredictable technical issues put buses out of service, so the tracking system keeps track of which vehicles are available for scheduling.

c. Staff Management

Trip operations include drivers and conductors who fulfill different duties during the trip. They assist the passengers to board safely and control the boarding process in other terminals. The assignment system provides the

flexibility of operations while keeping the definite record of the staff responsibilities for each journey.

d. Passenger Booking System

Travelers access the online booking platform to reserve seats for their desired trips. The same passenger can make a number of different journeys over time along the platform. The system records the booking date and time, booking class of passengers, their seat numbers, and the booking status that is either pending, confirmed or canceled.

e. Ticket Generation

When the booking is confirmed, the system automatically generates a ticket. This ticket specifies all the journey details, the fare value, and the issue date. The tickets have verification codes, which are checked by the staff during the boarding process.

f. Payment Processing

Flexibility in payment of tickets enables tickets to exist in the system before passengers complete their financial transactions. After a payment is made, the database stores the payment method used by the person, the payment amount, and the time. This method suitably supports the customer payment preferences, while keeping proper financial records.

g. Passenger Preferences

The individual travelers can have particular demands and preferences. Some passengers need extra luggage space, while others want SMS or email updates about their journeys. Travelers choose to receive notifications and get booking confirmations, future trip reminders, and alerts in case of schedule changes.

The operations of the company are based on an integrated database which links routes, buses, trips, staff, passengers, bookings, tickets and payments. This is a centralized design that allows the managers to see the current operations, enhance the coordination between different departments,

and enable efficient data management. The whole system allows SajiloYatra to provide reliable intercity transportation across Nepal.

1.3. Business Rules Derived from Operational Procedures

The following business rules define the constraints and structure of the database system of SajiloYatra, depending on the operational procedures of the company:

a. Each route should have at least one bus assigned to it before any trip can be placed on that route.

b. The number of confirmed bookings for any trip cannot be more than the number of seats available on the assigned bus.

c. Only a bus which is already assigned to the route of a trip can be used to operate the trip.

d. Buses which are not available for maintenance or operational reasons cannot be allocated to new trips.

e. Tickets are only generated when a booking status is changed to confirmed, and pending and canceled booking cannot generate tickets.

f. The tickets may be present in the system without payment, but payment should be made before the passenger boards.

g. The system supports three booking classes (Regular, Premium, and VIP) with different fare structures for passengers to choose from.

h. More than one staff can be allocated to each trip, such as drivers and conductors.

i. Travelers who have turned on travel notifications get notifications about their booking confirmations, trip reminders, and schedule changes.

j. Passengers who had confirmed bookings need to be informed when there is a change in the schedule of the trip due to operational or road issues.

k. Each passenger can travel on several trips throughout the time, while the system stores all the bookings.

l. Each passenger traveling on a trip must create an individual booking containing passenger details.

m. After payment is completed, the system generates a payment record containing the details of the payment method, amount, date, and transaction.

n. The system stores passenger preferences, including luggage requirements and travel notifications settings, to enhance service quality.

2. Initial ERD

2.1. Identification of Entities and Attributes (with Primary Keys)

The initial ERD of the SajiloYatra system contains four large entities that combine multiple related concepts into single tables. The four main entities in the initial ERD are Trip (Route_Trip_Bus_Staff), Booking, Passenger, and Ticket (Ticket_Payment). This structure is designed to show the common database design mistakes where distinct but related business concepts are stored together, resulting update, deletion, and insertion anomalies. In the initial ERD some attributes combine several related concepts. This does not follow normalization rules, so it is fixed later. The entities and their attributes, along with their primary keys, are described below:

Trip

Table 1: Trip Attributes

Attributes Name	Data Type	Size	Constraint
Trip_ID	NUMBER	5	Primary Key
Trip_Date	DATE	-	-
Departure_Time	TIMESTAMP	-	-
Arrival_Time	TIMESTAMP	-	-
Trip_Status	VARCHAR2	20	-
Route_ID	NUMBER	5	-
Route_Name	VARCHAR2	30	-
Start_Location	VARCHAR2	20	-
End_Location	VARCHAR2	20	-
Distance	NUMBER	6,2	-
Route_Status	VARCHAR2	20	-
Bus_ID	NUMBER	5	-
Bus_Number	VARCHAR2	20	-
Total_Capacity	NUMBER	3	-
Bus_Type	VARCHAR2	20	-
Bus_Status	VARCHAR2	20	-

Maintenance_Info	VARCHAR2	30	-
Staff_ID	NUMBER	5	-
Staff_Name	VARCHAR2	20	-
Contact	VARCHAR2	15	-
Role	VARCHAR2	20	-
Staff_Status	VARCHAR2	20	-

Passenger

Table 2: Passenger Attributes

Attributes Name	Data Type	Size	Constraint
Passenger_ID	NUMBER	5	Primary Key
Full_Name	VARCHAR2	20	-
Phone	VARCHAR2	15	-
Email	VARCHAR2	30	-
Luggage_Preference	VARCHAR2	20	-
Travel_Notification	VARCHAR2	20	-

Booking

Table 3: Booking Attributes

Attributes Name	Data Type	Size	Constraint
Booking_ID	NUMBER	5	Primary Key
Booking_Date	DATE	-	-
Booking_Class	VARCHAR2	20	-
Booking_Status	VARCHAR2	20	-

Ticket

Table 4: Ticket Attributes

Attributes Name	Data Type	Size	Constraint
Ticket_ID	NUMBER	5	Primary Key
Fare	NUMBER	8,2	-
Issue_Date	DATE	-	-
Ticket_Number	VARCHAR2	20	-
Ticket_Status	VARCHAR2	20	-

Payment_ID	NUMBER	5	-
Amount	NUMBER	8,2	-
Payment_Date	DATE	-	-
Payment_Method	VARCHAR	20	-
Payment_Status	VARCHAR2	20	-

2.1.1. List of Entities and Attributes (with Primary Keys)

Table 5: Entities and Attributes

Entity	**Attributes**	**Constraint**
Trip	Trip_ID, Trip_Date, Departure_Time, Arrival_Time, Trip_Status, Route_ID, Route_Name, Start_Location, End_Location, Distance, Route_Status, Bus_ID, Bus_Number, Total_Capacity, Bus_Type, Bus_Status, Maintenance_Info, Staff_ID, Staff_Name, Contact, Role, Staff_Status	Primary Key
Passenger	Passenger_ID, Full_Name, Phone, Email, Luggage_Preference, Travel_Notification	Primary Key
Booking	Booking_ID, Booking_Date, Booking_Class, Booking_Status	Primary Key
Ticket	Ticket_ID, Fare, Issue_Date, Ticket_Number, Ticket_Status, Payment_ID, Amount, Payment_Date, Payment_Method, Payment_Status	Primary Key

2.2. Initial Entity-Relationship Diagram (Before Normalization)

2.2.1. Relationship between Entities with Maximum and Minimum Cardinalities

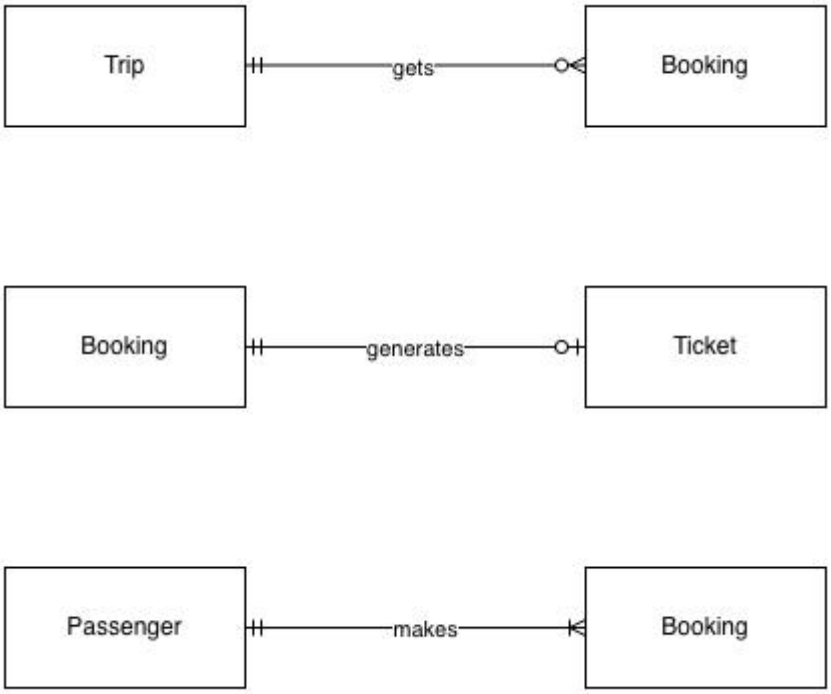

Figure 1: Relationship between Entities

Trip to Booking

i. Each trip gets zero or more bookings.
ii. Each booking is linked to a single trip.

Relationship: One (mandatory) Trip to Zero or Many (optional) Booking

Booking to Ticket

i. Each booking generates zero or one ticket.
ii. Each ticket is generated for a single booking.

Relationship: One (mandatory) Booking to Zero or One (optional) Ticket

Passenger to Booking

i. Each passenger makes one or many bookings.
ii. Each booking is assigned to a single passenger.

Relationship: One (mandatory) Passenger to One or Many (mandatory) Booking

2.2.2. Initial Entity Relationship Diagram

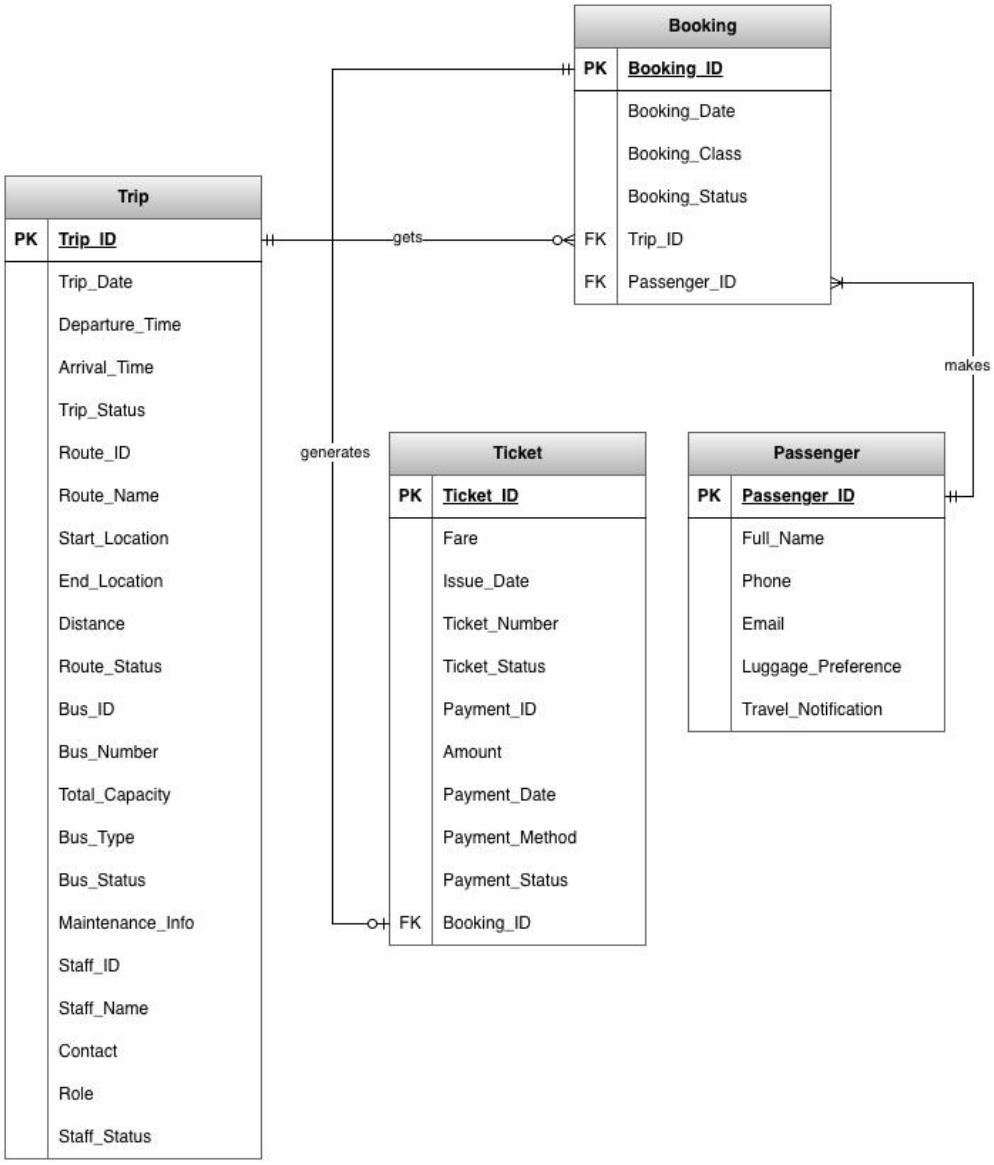

Figure 2: Initial Entity Relationship Diagram

3. Normalization

The process of managing a database to minimize unnecessary redundancy and dependency by breaking the data into logically related tables is called normalization. Normalization removes anomalies which may impact the data integrity and efficiency. Unless normalized, databases can experience duplication, inconsistencies, and other issues as they increase. It is a series of systemic steps that are used to create a reliable and well-organized database (GeeksforGeeks, 2025).

The anomalies handled by the normalization are (GeeksforGeeks, 2025):

i. Insertion Anomalies: It occurs when some data cannot be inserted due to the absence of required fields. For example, if a primary key is mandatory, but the value is not given, then the record cannot be inserted.

ii. Update Anomalies: This happens when updates in one record are not updated in other records creating inconsistencies. For example, updating the salary of an employee in one record but not in others may give wrong reports or calculations.

iii. Deletion Anomalies: This occurs when other important data is accidentally deleted when a record is being deleted. For example, the deletion of a customer record can also delete the order records of that customer.

Normalization process consists of several normal forms with a set of rules and conditions, which are (GeeksforGeeks, 2025):

i. First Normal Form (1NF): There should be no repetitive values of any attribute, with each attribute having atomic (single) values.

ii. Second Normal Form (2NF): It must be in 1NF form, and all the non-key attributes must be completely dependent on the primary key.

iii. Third Normal Form (3NF): It must be in 2NF and should not have any transitive dependencies among non key attributes.

iv. Boyce-Codd Normal Form (BCNF): The table must be in 3NF, and all the determinants should be a candidate key.

v. Fourth Normal Form (4NF): It has to be in BCNF and should not have multi-valued dependencies.

vi. Fifth Normal Form (5NF): The table is in 4NF and cannot be split further without losing data.

3.1. Unnormalized Form (UNF)

The database in its simplest form is known as Unnormalized Form. Tables in this form have repeating groups, redundant data, and may cause inefficiency and inconsistency. Due to these problems, UNF does not have the structure and reliability provided by the normalization process (GeeksforGeeks, 2025).

Identifying repeating group:

All the data are initially stored in a single table and repeating groups are identified in the unnormalized form. Repeating groups are the attributes that can have multiple values for a single record.

Trip (Trip_ID, Trip_Date, Departure_Time, Arrival_Time, Trip_Status, Route_ID, Route_Name, Start_Location, End_Location, Distance, Route_Status, Bus_ID, Bus_Number, Total_Capacity, Bus_Type, Bus_Status, Maintenance_Info, Staff_ID, Staff_Name, Contact, Role, Staff_Status, {Booking_ID, Booking_Date, Booking_Class, Booking_Status, Passenger_ID,

Full_Name, Phone, Email, Luggage_Preference, Travel_Notification, Ticket_ID, Fare, Issue_Date, Ticket_Number, Ticket_Status, Payment_ID, Amount, Payment_Date, Payment_Method, Payment_Status})

The initial database design consists of a single table called Trip, which is in the unnormalized form of a database. This table takes Trip_ID as the primary key and contains all information associated with the bus transportation system.

Basic information such as the Trip_Date, Departure_Time, Arrival_Time, and Trip_Status is stored in the table. The information about routes is recorded with the help of Route_ID, Route_Name, Start_Location, End_Location, Distance, and Route_Status. The data associated with the buses are Bus_ID, Bus_Number, Total_Capacity, Bus_Type, Bus_Status, and Maintenance_Info. The staff assignments are logged through Staff_ID, Staff_name, Contact, Role, and Staff_Status.

The only significant complexity is due to the repeating groups presented in curly braces. There can be multiple bookings per trip, and each booking includes embedded information regarding passengers, tickets and payments. The booking information consists of Booking_ID, Booking_Date, Booking_Class, and Booking_Status. The details of the passenger include Passenger_ID, Full_Name, Phone, Email, Luggage_Preference, and Travel_Notification. Tickets contain details such as, Ticket_ID, Fare, Issue_Date, Ticket_Number, and Ticket_Status. The details of the payment are monitored by Payment_ID, Amount, Payment_Date, Payment_Method, and Payment_Status.

The data in this unnormalized form has a high level of data redundancy since trip, route, bus and staff data is repeated with each booking. Repeating groups are against the basic principles of database design and cause possible anomalies during data operations. This structure is a base of normalization, which will break it down into several related tables to remove data redundancy and enhance data integrity.

3.2. First Normal Form (1NF)

When all the attributes are atomic, meaning each attribute has a single value and no repeating groups exist in any column, the database table is considered to be in first normal form (1NF) (Demba, 2013).

Requirements for 1NF (Demba, 2013):

i. Each column have the atomic values only.
ii. Each column contains values of same data type or domain.
iii. Each column has a unique name.

Separating repeating group:

In 1NF, repeating groups are separated and stored into different table, and all attributes are made atomic. To obtain 1NF, the unnormalized Trip table has been broken down into three different tables, which perform different functions in the database system.

Trip_1

(Trip_ID, Trip_Date, Departure_Time, Arrival_Time, Trip_Status, Route_ID, Route_Name, Start_Location, End_Location, Distance, Route_Status, Bus_ID, Bus_Number, Total_Capacity, Bus_Type, Bus_Status, Maintenance_Info, Staff_ID, Staff_Name, Contact, Role, Staff_Status)

Trip_1 is the first table that contains all the primary trip information with Trip_ID as the primary key. The table includes all the attributes that are directly associated with trips, routes, buses, and staff assignments. This table is in 1NF because it has no repeating groups.

Booking_1

(Booking_ID, Booking_Date, Booking_Class, Booking_Status, Passenger_ID, Full_Name, Phone, Email, Luggage_Preference, Travel_Notification, Ticket_ID, Fare, Issue_Date, Ticket_Number, Ticket_Status, Payment_ID, Amount, Payment_Date, Payment_Method, Payment_Status, Trip_ID*)

Booking_1 is the second table which contains all the information related to booking with Booking_ID as the primary key, and Trip_ID as the foreign key. The table contains all the booking-related information. This table is also in 1NF because it has no repeating groups.

3.3. Second Normal Form (2NF)

When a relation is in 1NF and all partial dependencies are removed, it is considered to be in second normal form (2NF). This means that all non-key attributes should rely on the full primary key, rather than just a subset of it. This is mostly applicable to tables that have composite primary keys (Demba, 2013).

Requirements for 2NF (Demba, 2013)**:**

i. The relation should already be in 1NF.
ii. All non-key attributes should be completely dependent on the entire primary key.
iii. There should be no partial dependencies.

Partial dependency happens when a table has a composite primary key and some non-key attributes depend on only part of that key, not the entire key. This may cause redundancy and inconsistencies in the data. Removing partial dependency makes all the non-key attributes totally dependent on the entire primary key enhancing the integrity and organization of the database (Demba, 2013).

Trip_2

(<u>Trip_ID</u>, Trip_Date, Departure_Time, Arrival_Time, Trip_Status, Route_ID, Route_Name, Start_Location, End_Location, Distance, Route_Status, Bus_ID, Bus_Number, Total_Capacity, Bus_Type, Bus_Status, Maintenance_Info, Staff_ID, Staff_Name, Contact, Role, Staff_Status)

Full Functional Dependency (FFD): Trip_ID → Trip_Date, Departure_Time, Arrival_Time, Trip_Status, Route_ID, Route_Name, Start_Location, End_Location, Distance, Route_Status, Bus_ID, Bus_Number, Total_Capacity, Bus_Type, Bus_Status, Maintenance_Info, Staff_ID, Staff_Name, Contact, Role, Staff_Status

Partial Functional Dependency (PFD): None (Trip_ID is single-attribute primary key, so no attributes depend on only part of a key).

Trip_2 is in 2NF because all non-key attributes depend fully on the primary key Trip_ID.

Booking_2

(Booking_ID, Booking_Date, Booking_Class, Booking_Status, Passenger_ID, Full_Name, Phone, Email, Luggage_Preference, Travel_Notification, Ticket_ID, Fare, Issue_Date, Ticket_Number, Ticket_Status, Payment_ID, Amount, Payment_Date, Payment_Method, Payment_Status, Trip_ID*)

Full Functional Dependency (FFD): Booking_ID → Booking_Date, Booking_Class, Booking_Status, Passenger_ID, Full_Name, Phone, Email, Luggage_Preference, Travel_Notification, Ticket_ID, Fare, Issue_Date, Ticket_Number, Ticket_Status, Payment_ID, Amount, Payment_Date, Payment_Method, Payment_Status, Trip_ID

Partial Functional Dependency (PFD): None (Booking_ID is single-attribute primary key, so no attributes depend on only part of a key).

Booking_2 is in 2NF because all non-key attributes depend fully on the primary key Booking_ID.

3.4. Third Normal Form (3NF)

When a table is in 2NF and no non-key attribute relies on another non-key attribute, it is considered to be in third normal form (3NF). Third normal form removes transitive dependencies among the non-key attributes (Demba, 2013).

Requirements for 3NF (Demba, 2013)**:**

i. The table should already be in 2NF.
ii. There should be no transitive dependencies.
iii. In all functional dependencies where X determines Y, either X is a superkey or Y is a prime attribute.

Transitive dependency happens when non-key attributes depend on another non-key attribute instead of the primary key. This also may cause redundancy and inconsistencies in the data. Removing transitive dependency ensures that all non-key attributes depend on the primary key, which enhances the database structure and integrity (Demba, 2013).

Transitive Dependency (TD): Trip_ID → Route_ID → Route_Name, Start_Location, End_Location, Distance, Route_Status

The above attributes are transitively dependent on Trip_ID through Route_ID, because Route_ID is a non-key attribute that uniquely identifies them.

Transitive Dependency (TD): Trip_ID → Bus_ID → Bus_Number, Total_Capacity, Bus_Type, Bus_Status, Maintenance_Info

The above attributes are transitively dependent on Trip_ID through Bus_ID, because Bus_ID is a non-key attribute that uniquely identifies them.

Transitive Dependency (TD): Trip_ID → Staff_ID → Staff_Name, Contact, Role, Staff_Status

The above attributes are transitively dependent on Trip_ID through Staff_ID, because Staff_ID is a non-key attribute that uniquely identifies them.

Transitive Dependency (TD): Booking_ID → Passenger_ID → Full_Name, Phone, Email, Luggage_Preference, Travel_Notification

The above attributes are transitively dependent on Booking_ID through Passenger_ID, because Passenger_ID is a non-key attribute that uniquely identifies them.

Transitive Dependency (TD): Booking_ID → Ticket_ID → Fare, Issue_Date, Ticket_Number, Ticket_Status

The above attributes are transitively dependent on Booking_ID through Ticket_ID, because Ticket_ID is a non-key attribute that uniquely identifies them.

Transitive Dependency (TD): Booking_ID → Payment_ID → Amount, Payment_Date, Payment_Method, Payment_Status

The above attributes are transitively dependent on Booking_ID through Payment_ID, because Payment_ID is a non-key attribute that uniquely identifies them.

Final 3NF:

Route_3 (Route_ID, Route_Name, Start_Location, End_Location, Distance, Route_Status)

Bus_3 (Bus_ID, Bus_Number, Total_Capacity, Bus_Type, Bus_Status, Maintenance_Info)

Staff_3 (Staff_ID, Staff_Name, Contact, Role, Staff_Status)

Trip_3 (Trip_ID, Trip_Date, Departure_Time, Arrival_Time, Trip_Status, Route_ID*, Bus_ID*)

Trip_Staff_3 (Trip_ID*, Staff_ID*)

Passenger_3 (Passenger_ID, Full_Name, Phone, Email, Luggage_Preference, Travel_Notification)

Booking_3 (Booking_ID, Booking_Date, Booking_Class, Booking_Status, Trip_ID*, Passenger_ID*)

Ticket_3 (Ticket_ID, Fare, Issue_Date, Ticket_Number, Ticket_Status, Booking_ID*)

Payment_3 (Payment_ID, Amount, Payment_Date, Payment_Method, Payment_Status, Ticket_ID*)

Transitive dependencies are removed in all tables, and all non-key attributes depend only on the primary key. That is why all the above tables are in 3NF.

4. Data Dictionary and Final ERD

4.1. List of Entities after Normalization

Route

Table 6: Route Table

Entity Name	Entity Description	Column Name	Column Description	Data Type	Size	Constraints
Route	Stores information about travel routes used by trips.	Route_ID	Unique identifier for route	NUMBER	5	PRIMARY KEY
		Route_Name	Name of the route	VARCHAR2	30	NOT NULL
		Start_Location	Starting location of route	VARCHAR2	20	NOT NULL, CHECK
		End_Location	Ending location of route	VARCHAR2	20	NOT NULL, CHECK
		Distance	Distance covered by route	NUMBER	6,2	NOT NULL, CHECK
		Route_Status	Availability status of route	VARCHAR2	20	NOT NULL, CHECK

Bus

Table 7: Bus Table

Entity Name	Entity Description	Column Name	Column Description	Data Type	Size	Constraints
Bus	Stores details of buses used for trips.	Bus_ID	Unique identifier for bus	NUMBER	5	PRIMARY KEY
		Bus_Number	Registration number of bus	VARCHAR2	20	UNIQUE, NOT NULL
		Total_Capacity	Total seating capacity of bus	NUMBER	3	NOT NULL, CHECK
		Bus_Type	Type of bus	VARCHAR2	20	NOT NULL, CHECK
		Bus_Status	Operational status	VARCHAR2	20	NOT NULL, CHECK

		Maintenance_ Info	Maintenance details	VARCHAR2	30	NULL

Staff

Table 8: Staff Table

Entity Name	Entity Description	Column Name	Column Description	Data Type	Size	Constraints
Staff	Stores information about staff operating trips.	Staff_ID	Unique staff identifier	NUMBER	5	PRIMARY KEY
		Staff_Name	Name of staff	VARCHAR2	20	NOT NULL
		Contact	Staff contact number	VARCHAR2	15	UNIQUE, NOT NULL
		Role	Staff role	VARCHAR2	20	NOT NULL, CHECK
		Staff_Status	Availability status of staff	VARCHAR2	20	NOT NULL, CHECK

Trip

Table 9: Trip Table

Entity Name	Entity Description	Column Name	Column Description	Data Type	Size	Constraints
Trip	Stores scheduled trip details.	Trip_ID	Unique trip identifier	NUMBER	5	PRIMARY KEY
		Trip_Date	Date of trip	DATE	-	NOT NULL
		Departure_ Time	Departure time	TIMESTAMP	-	NOT NULL
		Arrival_Time	Arrival time	TIMESTAMP	-	NOT NULL
		Trip_Status	Status of trip	VARCHAR2	20	NOT NULL, CHECK
		Route_ID	Route used for trip	NUMBER	5	NOT NULL, FOREIGN KEY
		Bus_ID	Bus assigned to trip	NUMBER	5	NOT NULL, FOREIGN KEY

Trip_Staff

Table 10: Trip_Staff Table

Entity Name	Entity Description	Column Name	Column Description	Data Type	Size	Constraints
Trip_ Staff	Solves many-to-many relationship	Trip_ID	Assigned trip	NUMBER	5	PRIMARY KEY (composite),

	between trip and staff.					FOREIGN KEY
		Staff_ID	Assigned staff	NUMBER	5	PRIMARY KEY (composite), FOREIGN KEY

Passenger

Table 11: Passenger Table

Entity Name	Entity Description	Column Name	Column Description	Data Type	Size	Constraints
Passenger	Stores passenger personal and preference details.	Passenger_ ID	Unique passenger identifier	NUMBER	5	PRIMARY KEY
		Full_Name	Passenger full name	VARCHAR2	20	NOT NULL
		Phone	Contact number of passenger	VARCHAR2	15	UNIQUE, NOY NULL
		Email	Email address of passenger	VARCHAR2	30	UNIQUE
		Luggage_ Preference	Passenger luggage requirement	VARCHAR2	20	CHECK
		Travel_ Notification	Notification preference	VARCHAR2	20	CHECK

Booking

Table 12: Booking Table

Entity Name	Entity Description	Column Name	Column Description	Data Type	Size	Constraints
Booking	Stores booking information made by passengers.	Booking_ID	Unique booking identifier	NUMBER	5	PRIMARY KEY
		Booking_Date	Date of booking	DATE	-	NOT NULL
		Booking_Class	Travel class	VARCHAR2	20	NOT NULL, CHECK
		Booking_ Status	Status of booking	VARCHAR2	20	NOT NULL, CHECK
		Trip_ID	Booked trip	NUMBER	5	NOT NULL, FOREIGN KEY
		Passenger_ID	Passenger making booking	NUMBER	5	NOT NULL, FOREIGN KEY

Ticket

Table 13: Ticket Table

Entity Name	Entity Description	Column Name	Column Description	Data Type	Size	Constraints
Ticket	Stores ticket details generated from bookings.	Ticket_ID	Unique ticket identifier	NUMBER	5	PRIMARY KEY
		Fare	Ticket fare	NUMBER	8,2	NOT NULL, CHECK
		Issue_Date	Ticket issue date	DATE	-	NOT NULL
		Ticket_ Number	Ticket number	VARCHAR2	20	UNIQUE, NOT NULL
		Ticket_Status	Status of ticket	VARCHAR2	20	NOT NULL, CHECK
		Booking_ID	Related booking	NUMBER	5	NOT NULL, FOREIGN KEY

Payment

Table 14: Payment Table

Entity Name	Entity Description	Column Name	Column Description	Data Type	Size	Constraints
Payment	Stores payment information for tickets.	Payment_ID	Unique payment identifier	NUMBER	5	PRIMARY KEY
		Amount	Payment amount	NUMBER	8,2	NOT NULL, CHECK
		Payment_Date	Date of payment	DATE	-	NULL
		Payment_ Method	Mode of payment	VARCHAR2	20	NULL, CHECK
		Payment_ Status	Status of payment	VARCHAR2	20	NOT NULL, CHECK
		Ticket_ID	Paid ticket	NUMBER	5	NOT NULL, FOREIGN KEY

4.2. Final ERD

This ERD shows SajiloYatra's bus reservation system where routes and buses are connected to scheduled trips, and staff members like drivers and conductors are assigned to these trips using the Trip_Staff table that links multiple staff to multiple trips. When a passengers book a trip, the system generates a ticket for them, and after they complete their payment, the transaction is recorded in the database, finishing the whole booking process.

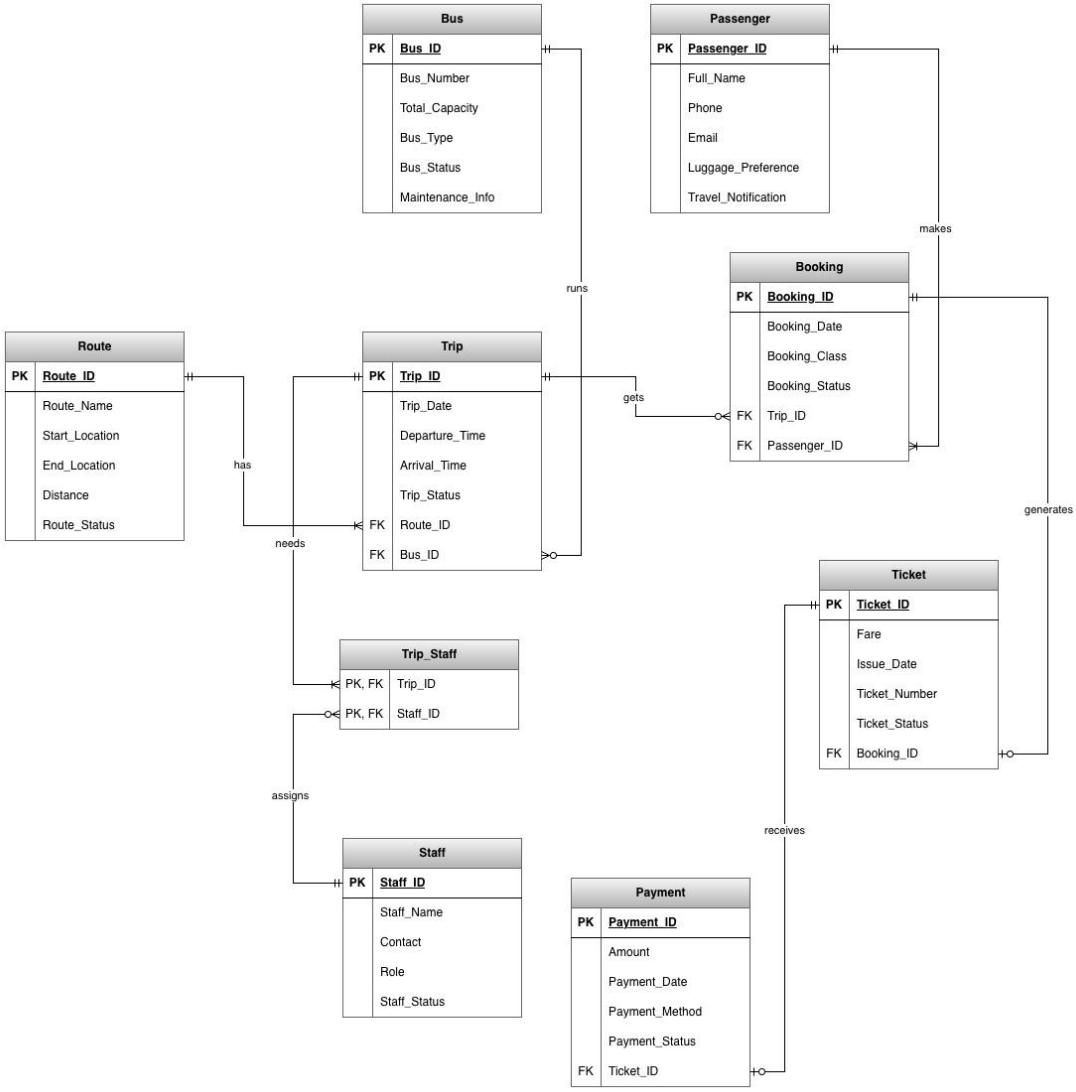

Figure 3: Final Entity Relationship Diagram

5. Database Implementation

5.1. Creating Relations and Tables

Passenger Table

```
SQL> CREATE TABLE Passenger (
  2  Passenger_ID NUMBER(5) PRIMARY KEY,
  3  Full_Name VARCHAR2(20) NOT NULL,
  4  Phone VARCHAR2(15) UNIQUE NOT NULL,
  5  Email VARCHAR2(30) UNIQUE,
  6  Luggage_Preference VARCHAR2(20) CHECK (Luggage_Preference IN ('Low', 'Medium', 'High')),
  7  Travel_Notification VARCHAR2(20) CHECK (Travel_Notification IN ('Yes', 'No'))
  8  );

Table created.

SQL> DESC Passenger;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 PASSENGER_ID                              NOT NULL NUMBER(5)
 FULL_NAME                                 NOT NULL VARCHAR2(20)
 PHONE                                     NOT NULL VARCHAR2(15)
 EMAIL                                              VARCHAR2(30)
 LUGGAGE_PREFERENCE                                 VARCHAR2(20)
 TRAVEL_NOTIFICATION                                VARCHAR2(20)
```

Figure 4: Passenger Table Created

Route Table

```
SQL> CREATE TABLE Route (
  2  Route_ID NUMBER(5) PRIMARY KEY,
  3  Route_Name VARCHAR2(30) NOT NULL,
  4  Start_Location VARCHAR2(20) NOT NULL CHECK (Start_Location IN ('Kathmandu', 'Pokhara', 'Chitwan', 'Butwal')),
  5  End_Location VARCHAR2(20) NOT NULL CHECK (End_Location IN ('Kathmandu', 'Pokhara', 'Chitwan', 'Butwal')),
  6  Distance NUMBER(6,2) NOT NULL CHECK (Distance > 0),
  7  Route_Status VARCHAR2(20) NOT NULL CHECK (Route_Status IN ('Active', 'Inactive', 'Under Maintenance')),
  8  CHECK (Start_Location != End_Location)
  9  );

Table created.

SQL> DESC Route;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 ROUTE_ID                                  NOT NULL NUMBER(5)
 ROUTE_NAME                                NOT NULL VARCHAR2(30)
 START_LOCATION                            NOT NULL VARCHAR2(20)
 END_LOCATION                              NOT NULL VARCHAR2(20)
 DISTANCE                                  NOT NULL NUMBER(6,2)
 ROUTE_STATUS                              NOT NULL VARCHAR2(20)
```

Figure 5: Route Table Created

Bus Table

```
SQL> CREATE TABLE Bus (
  2  Bus_ID NUMBER(5) PRIMARY KEY,
  3  Bus_Number VARCHAR2(20) UNIQUE NOT NULL,
  4  Total_Capacity NUMBER(3) NOT NULL CHECK (Total_Capacity > 0 AND Total_Capacity <= 100),
  5  Bus_Type VARCHAR2(20) NOT NULL CHECK (Bus_Type IN ('AC', 'Non-AC', 'Deluxe', 'VIP', 'Standard')),
  6  Bus_Status VARCHAR2(20) NOT NULL CHECK (Bus_Status IN ('Available', 'In Service', 'Maintenance', 'Unavailable')),
  7  Maintenance_Info VARCHAR2(30)
  8  );

Table created.

SQL> DESC Bus;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 BUS_ID                                    NOT NULL NUMBER(5)
 BUS_NUMBER                                NOT NULL VARCHAR2(20)
 TOTAL_CAPACITY                            NOT NULL NUMBER(3)
 BUS_TYPE                                  NOT NULL VARCHAR2(20)
 BUS_STATUS                                NOT NULL VARCHAR2(20)
 MAINTENANCE_INFO                                   VARCHAR2(30)
```

Figure 6: Bus Table Created

Staff Table

```
SQL> CREATE TABLE Staff (
  2  Staff_ID NUMBER(5) PRIMARY KEY,
  3  Staff_Name VARCHAR2(20) NOT NULL,
  4  Contact VARCHAR2(15) UNIQUE NOT NULL,
  5  Role VARCHAR2(20) NOT NULL CHECK (Role IN ('Driver', 'Conductor')),
  6  Staff_Status VARCHAR2(20) NOT NULL CHECK (Staff_Status IN ('Active', 'Inactive', 'On Leave'))
  7  );

Table created.

SQL> DESC Staff;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 STAFF_ID                                  NOT NULL NUMBER(5)
 STAFF_NAME                                NOT NULL VARCHAR2(20)
 CONTACT                                   NOT NULL VARCHAR2(15)
 ROLE                                      NOT NULL VARCHAR2(20)
 STAFF_STATUS                              NOT NULL VARCHAR2(20)
```

Figure 7: Staff Table Created

Trip Table

```
SQL> CREATE TABLE Trip (
  2  Trip_ID NUMBER(5) PRIMARY KEY,
  3  Trip_Date DATE NOT NULL,
  4  Departure_Time TIMESTAMP NOT NULL,
  5  Arrival_Time TIMESTAMP NOT NULL,
  6  Trip_Status VARCHAR2(20) NOT NULL CHECK (Trip_Status IN ('Scheduled', 'In Progress', 'Completed', 'Cancelled', 'Delayed')),
  7  Route_ID NUMBER(5) NOT NULL,
  8  Bus_ID NUMBER(5) NOT NULL,
  9  FOREIGN KEY(Route_ID) REFERENCES Route(Route_ID),
 10  FOREIGN KEY(Bus_ID) REFERENCES Bus(Bus_ID),
 11  CHECK (Arrival_Time > Departure_Time)
 12  );

Table created.

SQL> DESC Trip;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 TRIP_ID                                   NOT NULL NUMBER(5)
 TRIP_DATE                                 NOT NULL DATE
 DEPARTURE_TIME                            NOT NULL TIMESTAMP(6)
 ARRIVAL_TIME                              NOT NULL TIMESTAMP(6)
 TRIP_STATUS                               NOT NULL VARCHAR2(20)
 ROUTE_ID                                  NOT NULL NUMBER(5)
 BUS_ID                                    NOT NULL NUMBER(5)
```

Figure 8: Trip Table Created

Trip_Staff Table

```
SQL> CREATE TABLE Trip_Staff (
  2  Trip_ID NUMBER(5) NOT NULL,
  3  Staff_ID NUMBER(5) NOT NULL,
  4  PRIMARY KEY(Trip_ID, Staff_ID),
  5  FOREIGN KEY(Trip_ID) REFERENCES Trip(Trip_ID),
  6  FOREIGN KEY(Staff_ID) REFERENCES Staff(Staff_ID)
  7  );

Table created.

SQL> DESC Trip_Staff;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 TRIP_ID                                   NOT NULL NUMBER(5)
 STAFF_ID                                  NOT NULL NUMBER(5)
```

Figure 9: Trip_Staff Table Created

Booking Table

```
SQL> CREATE TABLE Booking (
  2  Booking_ID NUMBER(5) PRIMARY KEY,
  3  Booking_Date DATE NOT NULL,
  4  Booking_Class VARCHAR2(20) NOT NULL CHECK (Booking_Class IN ('Regular', 'Premium', 'VIP')),
  5  Booking_Status VARCHAR2(20) NOT NULL CHECK (Booking_Status IN ('Confirmed', 'Pending', 'Cancelled')),
  6  Trip_ID NUMBER(5) NOT NULL,
  7  Passenger_ID NUMBER(5) NOT NULL,
  8  FOREIGN KEY(Trip_ID) REFERENCES Trip(Trip_ID),
  9  FOREIGN KEY(Passenger_ID) REFERENCES Passenger(Passenger_ID)
 10  );

Table created.

SQL> DESC Booking;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 BOOKING_ID                                NOT NULL NUMBER(5)
 BOOKING_DATE                              NOT NULL DATE
 BOOKING_CLASS                             NOT NULL VARCHAR2(20)
 BOOKING_STATUS                            NOT NULL VARCHAR2(20)
 TRIP_ID                                   NOT NULL NUMBER(5)
 PASSENGER_ID                              NOT NULL NUMBER(5)
```

Figure 10: Booking Table Created

Ticket Table

```
SQL> CREATE TABLE Ticket (
  2  Ticket_ID NUMBER(5) PRIMARY KEY,
  3  Fare NUMBER(8,2) NOT NULL CHECK (Fare > 0),
  4  Issue_Date DATE NOT NULL,
  5  Ticket_Number VARCHAR2(20) UNIQUE NOT NULL,
  6  Ticket_Status VARCHAR2(20) NOT NULL CHECK (Ticket_Status IN ('Issued', 'Used', 'Cancelled', 'Expired')),
  7  Booking_ID NUMBER(5) NOT NULL,
  8  FOREIGN KEY(Booking_ID) REFERENCES Booking(Booking_ID)
  9  );

Table created.

SQL> DESC Ticket;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 TICKET_ID                                 NOT NULL NUMBER(5)
 FARE                                      NOT NULL NUMBER(8,2)
 ISSUE_DATE                                NOT NULL DATE
 TICKET_NUMBER                             NOT NULL VARCHAR2(20)
 TICKET_STATUS                             NOT NULL VARCHAR2(20)
 BOOKING_ID                                NOT NULL NUMBER(5)
```

Figure 11: Ticket Table Created

Payment Table

```
SQL> CREATE TABLE Payment (
  2  Payment_ID NUMBER(5) PRIMARY KEY,
  3  Amount NUMBER(8,2) NOT NULL CHECK (Amount > 0),
  4  Payment_Date DATE,
  5  Payment_Method VARCHAR2(20) CHECK (Payment_Method IN ('Cash', 'Card', 'Online')),
  6  Payment_Status VARCHAR2(20) NOT NULL CHECK (Payment_Status IN ('Paid', 'Unpaid', 'Pending', 'Refunded')),
  7  Ticket_ID NUMBER(5) NOT NULL,
  8  FOREIGN KEY(Ticket_ID) REFERENCES Ticket(Ticket_ID)
  9  );

Table created.

SQL> DESC Payment;
 Name                                      Null?    Type
 ----------------------------------------- -------- ----------------------------
 PAYMENT_ID                                NOT NULL NUMBER(5)
 AMOUNT                                    NOT NULL NUMBER(8,2)
 PAYMENT_DATE                                       DATE
 PAYMENT_METHOD                                     VARCHAR2(20)
 PAYMENT_STATUS                            NOT NULL VARCHAR2(20)
 TICKET_ID                                 NOT NULL NUMBER(5)
```

Figure 12: Passenger Table Created

5.2. Inserting Rows in Tables

Passenger

```
SQL> INSERT ALL
  2  INTO Passenger VALUES (1, 'Anjali Shrestha', '9812345671', 'anjali12@gmail.com', 'Medium', 'Yes')
  3  INTO Passenger VALUES (2, 'Anisha Khadka', '9889012342', 'anisha34@gmail.com', 'Low', 'Yes')
  4  INTO Passenger VALUES (3, 'Birat Rai', '9856789013', 'birat56@gmail.com', 'High', 'No')
  5  INTO Passenger VALUES (4, 'Sita Karki', '9823456784', 'sita78@gmail.com', 'Medium', 'Yes')
  6  INTO Passenger VALUES (5, 'Rewan Gurung', '9890123455', 'rewan90@gmail.com', 'Low', 'Yes')
  7  INTO Passenger VALUES (6, 'Ritesh Karki', '9867890126', 'ritesh12@gmail.com', 'Medium', 'No')
  8  INTO Passenger VALUES (7, 'Odisha Lama', '9834567897', 'odisha34@gmail.com', 'Low', 'Yes')
  9  SELECT * FROM DUAL;

7 rows created.

SQL> SELECT * FROM Passenger;

PASSENGER_ID FULL_NAME            PHONE           EMAIL                          LUGGAGE_PREFERENCE   TRAVEL_NOTIFICATION
------------ -------------------- --------------- ------------------------------ -------------------- -------------------
           1 Anjali Shrestha      9812345671      anjali12@gmail.com             Medium               Yes
           2 Anisha Khadka        9889012342      anisha34@gmail.com             Low                  Yes
           3 Birat Rai            9856789013      birat56@gmail.com              High                 No
           4 Sita Karki           9823456784      sita78@gmail.com               Medium               Yes
           5 Rewan Gurung         9890123455      rewan90@gmail.com              Low                  Yes
           6 Ritesh Karki         9867890126      ritesh12@gmail.com             Medium               No
           7 Odisha Lama          9834567897      odisha34@gmail.com             Low                  Yes

7 rows selected.
```

Figure 13: Rows Inserted into Passenger Table

Route

```
SQL> INSERT ALL
  2  INTO Route VALUES (1, 'Pokhara-Kathmandu', 'Pokhara', 'Kathmandu', 200, 'Active')
  3  INTO Route VALUES (2, 'Pokhara-Chitwan', 'Pokhara', 'Chitwan', 150, 'Active')
  4  INTO Route VALUES (3, 'Kathmandu-Butwal', 'Kathmandu', 'Butwal', 180, 'Active')
  5  INTO Route VALUES (4, 'Chitwan-Butwal', 'Chitwan', 'Butwal', 120, 'Active')
  6  INTO Route VALUES (5, 'Pokhara-Butwal', 'Pokhara', 'Butwal', 170, 'Active')
  7  INTO Route VALUES (6, 'Kathmandu-Chitwan', 'Kathmandu', 'Chitwan', 140, 'Active')
  8  INTO Route VALUES (7, 'Butwal-Pokhara', 'Butwal', 'Pokhara', 170, 'Inactive')
  9  SELECT * FROM DUAL;

7 rows created.

SQL> SELECT * FROM Route;

  ROUTE_ID ROUTE_NAME                   START_LOCATION       END_LOCATION           DISTANCE ROUTE_STATUS
---------- ---------------------------- -------------------- -------------------- ---------- --------------------
         1 Pokhara-Kathmandu            Pokhara              Kathmandu                   200 Active
         2 Pokhara-Chitwan              Pokhara              Chitwan                     150 Active
         3 Kathmandu-Butwal             Kathmandu            Butwal                      180 Active
         4 Chitwan-Butwal               Chitwan              Butwal                      120 Active
         5 Pokhara-Butwal               Pokhara              Butwal                      170 Active
         6 Kathmandu-Chitwan            Kathmandu            Chitwan                     140 Active
         7 Butwal-Pokhara               Butwal               Pokhara                     170 Inactive

7 rows selected.
```

Figure 14: Rows Inserted into Route Table

Bus

```
SQL> INSERT ALL
  2  INTO Bus VALUES (1, 'SY-101', 40, 'AC', 'Available', NULL)
  3  INTO Bus VALUES (2, 'SY-102', 50, 'Deluxe', 'Available', NULL)
  4  INTO Bus VALUES (3, 'SY-103', 30, 'Non-AC', 'Maintenance', 'Engine Repair')
  5  INTO Bus VALUES (4, 'SY-104', 45, 'AC', 'Available', NULL)
  6  INTO Bus VALUES (5, 'SY-105', 60, 'VIP', 'Available', NULL)
  7  INTO Bus VALUES (6, 'SY-106', 35, 'AC', 'Available', NULL)
  8  INTO Bus VALUES (7, 'SY-107', 40, 'Deluxe', 'Available', NULL)
  9  SELECT * FROM DUAL;

7 rows created.

SQL> SELECT * FROM Bus;

     BUS_ID BUS_NUMBER           TOTAL_CAPACITY BUS_TYPE            BUS_STATUS           MAINTENANCE_INFO
---------- -------------------- -------------- -------------------- -------------------- ------------------------------
         1 SY-101                             40 AC                   Available
         2 SY-102                             50 Deluxe               Available
         3 SY-103                             30 Non-AC               Maintenance          Engine Repair
         4 SY-104                             45 AC                   Available
         5 SY-105                             60 VIP                  Available
         6 SY-106                             35 AC                   Available
         7 SY-107                             40 Deluxe               Available

7 rows selected.
```

Figure 15: Rows Inserted into Bus Table

Staff

```
SQL> INSERT ALL
  2  INTO Staff VALUES (1, 'Raman Niraula', '9812345678', 'Driver', 'Active')
  3  INTO Staff VALUES (2, 'Simon Rai', '9823456789', 'Conductor', 'Active')
  4  INTO Staff VALUES (3, 'Himal Thapa', '9809876543', 'Driver', 'Active')
  5  INTO Staff VALUES (4, 'Miraj Gurung', '9811122233', 'Conductor', 'Active')
  6  INTO Staff VALUES (5, 'Ram Kumar Adhikari', '9809988776', 'Driver', 'Active')
  7  INTO Staff VALUES (6, 'Aakash Karki', '9812233445', 'Conductor', 'Active')
  8  INTO Staff VALUES (7, 'Kritan Tamang', '9803344556', 'Driver', 'Inactive')
  9  SELECT * FROM DUAL;

7 rows created.

SQL> SELECT * FROM Staff;

  STAFF_ID STAFF_NAME           CONTACT         ROLE                 STAFF_STATUS
---------- -------------------- --------------- -------------------- --------------------
         1 Raman Niraula        9812345678      Driver               Active
         2 Simon Rai            9823456789      Conductor            Active
         3 Himal Thapa          9809876543      Driver               Active
         4 Miraj Gurung         9811122233      Conductor            Active
         5 Ram Kumar Adhikari   9809988776      Driver               Active
         6 Aakash Karki         9812233445      Conductor            Active
         7 Kritan Tamang        9803344556      Driver               Inactive

7 rows selected.
```

Figure 16: Rows Inserted into Staff Table

Trip

```
SQL> INSERT ALL
  2  INTO Trip VALUES (1, DATE '2025-12-28', TIMESTAMP '2025-12-28 07:00:00', TIMESTAMP '2025-12-28 13:00:00', 'Scheduled', 1, 1)
  3  INTO Trip VALUES (2, DATE '2025-12-29', TIMESTAMP '2025-12-29 09:00:00', TIMESTAMP '2025-12-29 15:00:00', 'Scheduled', 1, 2)
  4  INTO Trip VALUES (3, DATE '2025-12-28', TIMESTAMP '2025-12-28 06:00:00', TIMESTAMP '2025-12-28 12:00:00', 'Scheduled', 2, 4)
  5  INTO Trip VALUES (4, DATE '2025-12-30', TIMESTAMP '2025-12-30 08:00:00', TIMESTAMP '2025-12-30 14:00:00', 'Scheduled', 5, 7)
  6  INTO Trip VALUES (5, DATE '2025-12-31', TIMESTAMP '2025-12-31 11:00:00', TIMESTAMP '2025-12-31 17:00:00', 'Scheduled', 6, 2)
  7  INTO Trip VALUES (6, DATE '2025-01-15', TIMESTAMP '2025-01-15 10:00:00', TIMESTAMP '2025-01-15 16:00:00', 'Completed', 3, 5)
  8  INTO Trip VALUES (7, DATE '2025-01-11', TIMESTAMP '2025-01-11 14:00:00', TIMESTAMP '2025-01-11 20:00:00', 'Completed', 4, 6)
  9  SELECT * FROM DUAL;

7 rows created.

SQL> SELECT * FROM Trip;

   TRIP_ID TRIP_DATE DEPARTURE_TIME                                                      ARRIVAL_TIME                                                          TRIP_STATUS        ROUTE_ID     BUS_ID
---------- --------- ------------------------------------------------------------------ ------------------------------------------------------------------ -------------------- ---------- ----------
         1 28-DEC-25 28-DEC-25 07.00.00.000000 AM                                       28-DEC-25 01.00.00.000000 PM                                          Scheduled             1          1
         2 29-DEC-25 29-DEC-25 09.00.00.000000 AM                                       29-DEC-25 03.00.00.000000 PM                                          Scheduled             1          2
         3 28-DEC-25 28-DEC-25 06.00.00.000000 AM                                       28-DEC-25 12.00.00.000000 PM                                          Scheduled             2          4
         4 30-DEC-25 30-DEC-25 08.00.00.000000 AM                                       30-DEC-25 02.00.00.000000 PM                                          Scheduled             5          7
         5 31-DEC-25 31-DEC-25 11.00.00.000000 AM                                       31-DEC-25 05.00.00.000000 PM                                          Scheduled             6          2
         6 15-JAN-25 15-JAN-25 10.00.00.000000 AM                                       15-JAN-25 04.00.00.000000 PM                                          Completed             3          5
         7 11-JAN-25 11-JAN-25 02.00.00.000000 PM                                       11-JAN-25 08.00.00.000000 PM                                          Completed             4          6

7 rows selected.
```

Figure 17: Rows Inserted into Trip Table

Trip_Staff

```
SQL> INSERT ALL
  2  INTO Trip_Staff VALUES (1, 1)
  3  INTO Trip_Staff VALUES (1, 2)
  4  INTO Trip_Staff VALUES (2, 3)
  5  INTO Trip_Staff VALUES (2, 4)
  6  INTO Trip_Staff VALUES (3, 5)
  7  INTO Trip_Staff VALUES (3, 6)
  8  INTO Trip_Staff VALUES (4, 1)
  9  INTO Trip_Staff VALUES (4, 2)
 10  INTO Trip_Staff VALUES (5, 3)
 11  INTO Trip_Staff VALUES (5, 4)
 12  INTO Trip_Staff VALUES (6, 5)
 13  INTO Trip_Staff VALUES (6, 6)
 14  INTO Trip_Staff VALUES (7, 1)
 15  INTO Trip_Staff VALUES (7, 2)
 16  SELECT * FROM DUAL;

14 rows created.

SQL> SELECT * FROM Trip_Staff;

   TRIP_ID   STAFF_ID
---------- ---------
         1         1
         1         2
         2         3
         2         4
         3         5
         3         6
         4         1
         4         2
         5         3
         5         4
         6         5
         6         6
         7         1
         7         2

14 rows selected.
```

Figure 18: Rows Inserted into Trip_Staff Table

Booking

```
SQL> INSERT ALL
  2  INTO Booking VALUES (1, DATE '2025-12-01', 'Regular', 'Confirmed', 1, 1)
  3  INTO Booking VALUES (2, DATE '2025-12-02', 'VIP', 'Confirmed', 2, 1)
  4  INTO Booking VALUES (3, DATE '2025-12-03', 'Premium', 'Confirmed', 3, 1)
  5  INTO Booking VALUES (4, DATE '2025-12-05', 'Regular', 'Confirmed', 1, 2)
  6  INTO Booking VALUES (5, DATE '2025-12-10', 'VIP', 'Confirmed', 2, 2)
  7  INTO Booking VALUES (6, DATE '2025-12-12', 'Premium', 'Confirmed', 3, 5)
  8  INTO Booking VALUES (7, DATE '2025-01-02', 'Regular', 'Confirmed', 6, 3)
  9  INTO Booking VALUES (8, DATE '2025-01-03', 'VIP', 'Confirmed', 7, 3)
 10  INTO Booking VALUES (9, DATE '2025-12-20', 'Regular', 'Confirmed', 4, 4)
 11  SELECT * FROM DUAL;

9 rows created.

SQL> SELECT * FROM Booking;

BOOKING_ID BOOKING_D BOOKING_CLASS        BOOKING_STATUS         TRIP_ID PASSENGER_ID
---------- --------- -------------------- -------------------- ---------- ------------
         1 01-DEC-25 Regular              Confirmed                     1            1
         2 02-DEC-25 VIP                  Confirmed                     2            1
         3 03-DEC-25 Premium              Confirmed                     3            1
         4 05-DEC-25 Regular              Confirmed                     1            2
         5 10-DEC-25 VIP                  Confirmed                     2            2
         6 12-DEC-25 Premium              Confirmed                     3            5
         7 02-JAN-25 Regular              Confirmed                     6            3
         8 03-JAN-25 VIP                  Confirmed                     7            3
         9 20-DEC-25 Regular              Confirmed                     4            4

9 rows selected.
```

Figure 19: Rows Inserted into Booking Table

Ticket

```
SQL> INSERT ALL
  2  INTO Ticket VALUES (1, 1200, DATE '2025-12-01', 'T001', 'Issued', 1)
  3  INTO Ticket VALUES (2, 1500, DATE '2025-12-02', 'T002', 'Issued', 2)
  4  INTO Ticket VALUES (3, 1300, DATE '2025-12-03', 'T003', 'Issued', 3)
  5  INTO Ticket VALUES (4, 1400, DATE '2025-12-05', 'T004', 'Issued', 4)
  6  INTO Ticket VALUES (5, 1250, DATE '2025-12-10', 'T005', 'Issued', 5)
  7  INTO Ticket VALUES (6, 1500, DATE '2025-12-12', 'T006', 'Issued', 6)
  8  INTO Ticket VALUES (7, 1600, DATE '2025-01-02', 'T007', 'Used', 7)
  9  INTO Ticket VALUES (8, 1700, DATE '2025-01-03', 'T008', 'Used', 8)
 10  INTO Ticket VALUES (9, 1800, DATE '2025-12-20', 'T009', 'Issued', 9)
 11  SELECT * FROM DUAL;

9 rows created.

SQL> SELECT * FROM Ticket;

 TICKET_ID       FARE ISSUE_DAT TICKET_NUMBER        TICKET_STATUS        BOOKING_ID
---------- ---------- --------- -------------------- -------------------- ----------
         1       1200 01-DEC-25 T001                 Issued                        1
         2       1500 02-DEC-25 T002                 Issued                        2
         3       1300 03-DEC-25 T003                 Issued                        3
         4       1400 05-DEC-25 T004                 Issued                        4
         5       1250 10-DEC-25 T005                 Issued                        5
         6       1500 12-DEC-25 T006                 Issued                        6
         7       1600 02-JAN-25 T007                 Used                          7
         8       1700 03-JAN-25 T008                 Used                          8
         9       1800 20-DEC-25 T009                 Issued                        9

9 rows selected.
```

Figure 20: Rows Inserted into Ticket Table

Payment

```
SQL> INSERT ALL
  2  INTO Payment VALUES (1, 1200, DATE '2025-12-01', 'Cash', 'Paid', 1)
  3  INTO Payment VALUES (2, 1500, DATE '2025-12-02', 'Card', 'Paid', 2)
  4  INTO Payment VALUES (3, 1300, DATE '2025-12-03', 'Cash', 'Paid', 3)
  5  INTO Payment VALUES (4, 1400, DATE '2025-12-05', 'Online', 'Paid', 4)
  6  INTO Payment VALUES (5, 1250, DATE '2025-12-10', 'Card', 'Paid', 5)
  7  INTO Payment VALUES (6, 1500, DATE '2025-12-12', 'Card', 'Paid', 6)
  8  INTO Payment VALUES (7, 1600, NULL, NULL, 'Unpaid', 7)
  9  INTO Payment VALUES (8, 1700, DATE '2025-01-03', 'Cash', 'Paid', 8)
 10  INTO Payment VALUES (9, 1800, DATE '2025-12-20', 'Online', 'Paid', 9)
 11  SELECT * FROM DUAL;

 9 rows created.

SQL> SELECT * FROM Payment;

 PAYMENT_ID     AMOUNT PAYMENT_D PAYMENT_METHOD       PAYMENT_STATUS       TICKET_ID
 ---------- ---------- --------- -------------------- -------------------- ---------
          1       1200 01-DEC-25 Cash                 Paid                         1
          2       1500 02-DEC-25 Card                 Paid                         2
          3       1300 03-DEC-25 Cash                 Paid                         3
          4       1400 05-DEC-25 Online               Paid                         4
          5       1250 10-DEC-25 Card                 Paid                         5
          6       1500 12-DEC-25 Card                 Paid                         6
          7       1600                                Unpaid                       7
          8       1700 03-JAN-25 Cash                 Paid                         8
          9       1800 20-DEC-25 Online               Paid                         9

 9 rows selected.
```

Figure 21: Rows Inserted into Payment Table

6. Database Quering

6.1. Information Query

Question 1

```
SQL> SELECT r.Route_ID, r.Route_Name, COUNT(t.Trip_ID) AS Total_Trips
  2  FROM Route r
  3  LEFT JOIN Trip t ON r.Route_ID = t.Route_ID
  4  GROUP BY r.Route_ID, r.Route_Name
  5  ORDER BY r.Route_ID;

  ROUTE_ID ROUTE_NAME                     TOTAL_TRIPS
---------- ------------------------------ -----------
         1 Pokhara-Kathmandu                        2
         2 Pokhara-Chitwan                          1
         3 Kathmandu-Butwal                         1
         4 Chitwan-Butwal                           1
         5 Pokhara-Butwal                           1
         6 Kathmandu-Chitwan                        1
         7 Butwal-Pokhara                           0

7 rows selected.
```

Figure 22: Information Query 1

This query shows all available routes and the number of trips which are scheduled on each route.

Question 2

```
SQL> SELECT t.Trip_ID, t.Departure_Time, r.Route_Name
  2  FROM Trip t
  3  JOIN Route r ON t.Route_ID = r.Route_ID
  4  WHERE r.Start_Location = 'Pokhara'
  5  ORDER BY t.Departure_Time;

   TRIP_ID DEPARTURE_TIME                                                       ROUTE_NAME
---------- ---------------------------------------------------------------------- ------------------------------
         3 28-DEC-25 06.00.00.000000 AM                                          Pokhara-Chitwan
         1 28-DEC-25 07.00.00.000000 AM                                          Pokhara-Kathmandu
         2 29-DEC-25 09.00.00.000000 AM                                          Pokhara-Kathmandu
         4 30-DEC-25 08.00.00.000000 AM                                          Pokhara-Butwal
```

Figure 23: Information Query 2

This query displays all the trips that start from Pokhara, their departure time, and their route name.

Question 3

```
SQL> SELECT p.Full_Name, COUNT(b.Booking_ID) AS Total_Trips
  2  FROM Passenger p
  3  JOIN Booking b ON p.Passenger_ID = b.Passenger_ID
  4  WHERE p.Full_Name LIKE 'A%'
  5  GROUP BY p.Passenger_ID, p.Full_Name
  6  ORDER BY p.Full_Name;

FULL_NAME            TOTAL_TRIPS
-------------------- -----------
Anisha Khadka                  2
Anjali Shrestha                3
```

Figure 24: Information Query 3

This query lists passengers whose names start with ‘A’ and shows how many trips each of them has booked.

Question 4

```
SQL> SELECT p.Passenger_ID, p.Full_Name, COUNT(b.Booking_ID) AS Total_Bookings
  2  FROM Passenger p
  3  JOIN Booking b ON p.Passenger_ID = b.Passenger_ID
  4  WHERE b.Booking_Date BETWEEN DATE '2025-12-01' AND DATE '2025-12-31'
  5  GROUP BY p.Passenger_ID, p.Full_Name
  6  HAVING COUNT(b.Booking_ID) > 2
  7  ORDER BY Total_Bookings DESC;

PASSENGER_ID FULL_NAME            TOTAL_BOOKINGS
------------ -------------------- --------------
           1 Anjali Shrestha                    3
```

Figure 25: Information Query 4

This query finds out those passengers who have booked more than two trips in December 2025.

Question 5

```
SQL> SELECT t.Ticket_ID, p.Full_Name, b.Trip_ID,
  2  CASE
  3  WHEN pay.Payment_Status IS NULL THEN 'Unpaid'
  4  ELSE pay.Payment_Status
  5  END AS Payment_Status
  6  FROM Ticket t
  7  JOIN Booking b ON t.Booking_ID = b.Booking_ID
  8  JOIN Passenger p ON b.Passenger_ID = p.Passenger_ID
  9  LEFT JOIN Payment pay ON t.Ticket_ID = pay.Ticket_ID
 10  WHERE t.Issue_Date BETWEEN DATE '2025-01-01' AND DATE '2025-01-31'
 11  ORDER BY t.Ticket_ID;

 TICKET_ID FULL_NAME               TRIP_ID PAYMENT_STATUS
---------- -------------------- ---------- --------------------
         7 Birat Rai                     6 Unpaid
         8 Birat Rai                     7 Paid
```

Figure 26: Information Query 5

This query displays the names and trip IDs of those passengers who purchased tickets in January 2025 with their payment status.

6.2. Transaction Query

Question 1

```
SQL> SELECT * FROM (
  2  SELECT * FROM Trip
  3  ORDER BY Departure_Time DESC
  4  )
  5  WHERE ROWNUM = 1;

   TRIP_ID TRIP_DATE DEPARTURE_TIME                                         ARRIVAL_TIME                                                TRIP_STATUS           ROUTE_ID     BUS_ID
---------- --------- ---------------------------------------------------------------------------------------------------------------------- ---------------------- ---------- ----------
         5 31-DEC-25 31-DEC-25 11.00.00.000000 AM                           31-DEC-25 05.00.00.000000 PM                                Scheduled                   6          2
```

Figure 27: Transaction Query 1

This query finds the trip that has the latest departure time.

Question 2

```
SQL> SELECT * FROM (
  2  SELECT p.Passenger_ID, p.Full_Name, COUNT(b.Booking_ID) AS Total_Trips
  3  FROM Passenger p
  4  JOIN Booking b ON p.Passenger_ID = b.Passenger_ID
  5  GROUP BY p.Passenger_ID, p.Full_Name
  6  ORDER BY COUNT(b.Booking_ID) DESC
  7  )
  8  WHERE ROWNUM <= 3;

PASSENGER_ID FULL_NAME            TOTAL_TRIPS
------------ -------------------- -----------
           1 Anjali Shrestha               3
           3 Birat Rai                     2
           2 Anisha Khadka                 2
```

Figure 28: Transaction Query 2

This query lists the top three passengers who have booked the highest number of trips.

Question 3

```
SQL> SELECT p.Passenger_ID, p.Full_Name, SUM(pay.Amount) AS Total_Paid
  2  FROM Passenger p
  3  JOIN Booking b ON p.Passenger_ID = b.Passenger_ID
  4  JOIN Ticket t ON b.Booking_ID = t.Booking_ID
  5  JOIN Payment pay ON t.Ticket_ID = pay.Ticket_ID
  6  WHERE pay.Payment_Status = 'Paid'
  7  GROUP BY p.Passenger_ID, p.Full_Name
  8  HAVING SUM(pay.Amount) > (SELECT AVG(Amount) FROM Payment WHERE Payment_Status = 'Paid')
  9  ORDER BY Total_Paid DESC;

PASSENGER_ID FULL_NAME            TOTAL_PAID
------------ -------------------- ----------
           1 Anjali Shrestha            4000
           2 Anisha Khadka              2650
           4 Sita Karki                 1800
           3 Birat Rai                  1700
           5 Rewan Gurung               1500
```

Figure 29: Transaction Query 3

This query displays the total amount paid by the passengers who have paid more than the average amount.

Question 4

```
SQL> SELECT r.Route_ID, r.Route_Name, COUNT(DISTINCT t.Trip_ID) AS Total_Trips, SUM(pay.Amount) AS Total_Revenue
  2  FROM Route r
  3  JOIN Trip t ON r.Route_ID = t.Route_ID
  4  JOIN Booking b ON t.Trip_ID = b.Trip_ID
  5  JOIN Ticket tk ON b.Booking_ID = tk.Booking_ID
  6  JOIN Payment pay ON tk.Ticket_ID = pay.Ticket_ID
  7  WHERE pay.Payment_Status = 'Paid'
  8  GROUP BY r.Route_ID, r.Route_Name
  9  ORDER BY Total_Revenue DESC;

  ROUTE_ID ROUTE_NAME                      TOTAL_TRIPS TOTAL_REVENUE
---------- ------------------------------ ----------- -------------
         1 Pokhara-Kathmandu                        2          5350
         2 Pokhara-Chitwan                          1          2800
         5 Pokhara-Butwal                           1          1800
         4 Chitwan-Butwal                           1          1700
```

Figure 30: Transaction Query 4

This query displays the total number of trips made on a route along with the total revenue.

Question 5

```
SQL> SELECT t.Trip_ID, COUNT(b.Booking_ID) AS Total_Bookings, bus.Total_Capacity
  2  FROM Trip t
  3  JOIN Bus bus ON t.Bus_ID = bus.Bus_ID
  4  LEFT JOIN Booking b ON t.Trip_ID = b.Trip_ID
  5  GROUP BY t.Trip_ID, bus.Total_Capacity
  6  HAVING COUNT(b.Booking_ID) < (0.5 * bus.Total_Capacity)
  7  ORDER BY Total_Bookings ASC;

   TRIP_ID TOTAL_BOOKINGS TOTAL_CAPACITY
---------- -------------- --------------
         5              0             50
         7              1             35
         6              1             60
         4              1             40
         3              2             45
         2              2             50
         1              2             40

7 rows selected.
```

Figure 31: Transaction Query 5

This query shows trips where fewer than half the bus seats got booked.

7. Critical Evaluation

7.1. Critical Evaluation of Module, Its Usage and Relation with Other Subject

The Database module has been very useful and practical. Initially, it was thought that databases simply involved storage of data, but the module showed how databases actually organize information logically and efficiently. The concepts learned (normalization, relationships, SQL) have changed the approach to structuring any kind of information.

The most surprising fact was the extent to which this is related to other subjects. In Software Engineering, requirements gathering and system design were learned, which is exactly what was done here but for data. The connection to programming is also clear as all applications require a database backend, and the knowledge of SQL helps in writing better code. Even the complex queries written are basically what is needed for data analysis work.

Using Oracle SQL Plus was challenging but worthwhile since the tool is industry standard. The emphasis on constraints and referential integrity showed why some real applications have data problems, which is often poor database design. In the end, this provides the basic knowledge required for development, analysis, and design.

7.2. Critical Assessment of Coursework

The SajiloYatra project provided realistic and comprehensive learning experience. The case study felt realistic that addresses real business issues, such as booking and payments. The step by step format was appreciable, beginning with ERD, normalization, and then SQL implementation. Each part was built on the previous one, which contributed to strengthening ideas.

Multiple challenges occurred during development. The most challenging task was identifying the entities and distinguishing them from attributes. Normalization to Third Normal Form without losing the data integrity required a lot of effort. Developing standard test data among the related tables was time-consuming, and the Oracle syntax had a few challenges with dates and constraints.

Practical database skills were developed through this coursework. Through practice, complex SQL queries were easily manageable. Proper database design clearly prevents data anomalies and improves efficiency. The most important thing is that database development need not only technical expertise but also business knowledge. This project brought theory and practice together, which helped to achieve genuine confidence in database work.

8. Drop Query and Database Dump file Creation

8.1. Drop Query

Passenger Table Drop

```
SQL> DROP TABLE Passenger;
Table dropped.
```

Figure 32: Passenger Table Dropped

Route Table Drop

```
SQL> DROP TABLE Route;
Table dropped.
```

Figure 33: Route Table Dropped

Bus Table Drop

```
SQL> DROP TABLE Bus;
Table dropped.
```

Figure 34: Bus Table Dropped

Staff Table Drop

```
SQL> DROP TABLE Staff;
Table dropped.
```

Figure 35: Staff Table Dropped

Trip Table Drop

```
SQL> DROP TABLE Trip;
Table dropped.
```

Figure 36: Trip Table Dropped

Trip_Staff Table Drop

```
SQL> DROP TABLE Trip_Staff;
Table dropped.
```

Figure 37: Trip_Staff Table Dropped

Booking Table Drop

```
SQL> DROP TABLE Booking;
Table dropped.
```

Figure 38: Booking Table Dropped

Ticket Table Drop

```
SQL> DROP TABLE Ticket;

Table dropped.
```

Figure 39: Ticket Table Dropped

Payment Table Drop

```
SQL> DROP TABLE Payment;

Table dropped.
```

Figure 40: Payment Table Dropped

8.2. Dump file Creation

A dump file is created to backup the entire database structure and data in one file that can be easily restored and tested on another system.

```
[root@582c75ef35e3 /]# exp SadikshyaKarki/24046034 file=SadikshyaKarki.dmp

Export: Release 11.2.0.1.0 - Production on Tue Jan 13 14:31:53 2026

Copyright (c) 1982, 2009, Oracle and/or its affiliates.  All rights reserved.


Connected to: Oracle Database 11g Enterprise Edition Release 11.2.0.1.0 - 64bit Production
With the Partitioning, OLAP, Data Mining and Real Application Testing options
Export done in US7ASCII character set and AL16UTF16 NCHAR character set
server uses AL32UTF8 character set (possible charset conversion)
. exporting pre-schema procedural objects and actions
. exporting foreign function library names for user SADIKSHYAKARKI
. exporting PUBLIC type synonyms
. exporting private type synonyms
. exporting object type definitions for user SADIKSHYAKARKI
About to export SADIKSHYAKARKI's objects ...
. exporting database links
. exporting sequence numbers
. exporting cluster definitions
. about to export SADIKSHYAKARKI's tables via Conventional Path ...
. . exporting table                       BOOKING          9 rows exported
. . exporting table                           BUS          7 rows exported
. . exporting table                     PASSENGER          7 rows exported
. . exporting table                       PAYMENT          9 rows exported
. . exporting table                         ROUTE          7 rows exported
. . exporting table                         STAFF          7 rows exported
. . exporting table                        TICKET          9 rows exported
. . exporting table                          TRIP          7 rows exported
. . exporting table                    TRIP_STAFF         14 rows exported
. exporting synonyms
. exporting views
. exporting stored procedures
. exporting operators
. exporting referential integrity constraints
. exporting triggers
. exporting indextypes
. exporting bitmap, functional and extensible indexes
. exporting posttables actions
. exporting materialized views
. exporting snapshot logs
. exporting job queues
. exporting refresh groups and children
. exporting dimensions
. exporting post-schema procedural objects and actions
. exporting statistics
Export terminated successfully without warnings.
[root@582c75ef35e3 /]# ls
SadikshyaKarki.dmp  bin  boot  dev  entrypoint.sh  etc  home  install  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  u01  usr  var
```

Figure 41: Dump File Created

9. Conclusion

The SajiloYatra database system was developed to solve real problems experienced by intercity travel services in Nepal. In this project, a thorough database solution was created that handles the passengers, trips, routes, buses, tickets and payments effectively. The systematic approach of collecting the requirements in the first phase up to the normalization phase and finally to the final implementation stage in the Oracle SQL database demonstrated how proper database design solves business problems.

The normalization process was done to guarantee the integrity of the data by removing redundancy an organizing data into Third Normal Form. The final design has eight tables that connect logically with proper relationships and rules built right into the database structure. The Oracle SQL Plus implementation was successful in creating all tables with corresponding primary keys, foreign keys and constraints to keep data consistent. The inserted test data covered various scenarios, and the SQL queries written can be used to retrieve both simple information and complex transaction reports. The system is capable of tracking bookings, calculating revenue, monitoring capacity, and generating valuable business information.

Altogether, the SajiloYatra database provides a strong base for managing intercity travel operations. The design is flexible enough to support future growth without compromising data consistency and integrity in all the operations.

10. References

Demba, M. (2013) ALGORITHM FOR RELATIONAL DATABASE NORMALIZATION UP TO 3NF. *International Journal of Database Management Systems*, 5, pp.39-51.

GeeksforGeeks. (2025) *GeeksforGeeks* [Online]. Available from: https://www.geeksforgeeks.org/dbms/introduction-of-database-normalization/ [Accessed 15 December 2025].

GeeksforGeeks. (2025) *GeeksforGeeks* [Online]. Available from: https://www.geeksforgeeks.org/dbms/types-of-normal-forms-in-dbms/ [Accessed 15 December 2025].

Silberschatz, A., Korth, H.F. & Sudarshan, S. (2020) *Database System Concepts*. 7th ed. New York: McGraw-Hill Educatio.